

An Introduction to MicroPython



The Python language has been made lean and fast to run on microcontrollers. Beginners and experts can use it to control their electronics projects with ease.

The MicroPython project is an open source implementation of Python 3 that includes a small subset of the Python standard libraries, and is optimised to run on microcontrollers with constrained environments like limited ROM, RAM and processing power. It came about after a successful Kick-starter campaign by Damien George.

What is MicroPython?

MicroPython includes the standard Python interpreter that runs on small embedded development boards. With MicroPython, you can write Python scripts to control hardware. For example, you can make LEDs blink, communicate with a temperature sensor, control motors and publish a sensor reading on the Internet. It is worth noting that the firmware for such embedded devices is generally coded in Assembly, C or C++, but using MicroPython you can achieve almost the same results as with high level Python.

What MicroPython is *not*

Unlike the desktop version of Python, MicroPython is a stripped down version for microcontrollers; therefore, it doesn't support all the Python libraries and functionalities. In the world of microcontrollers, the firmware is directly burned to the ROM (also known as program memory), and there is no file system. MicroPython implements a minimal type of file system directly on the Flash memory of the microcontrollers. If the device has 1MB or more of storage, then it will be set

up (upon first boot) to contain a file system. This file system uses the FAT format and is stored in the Flash memory by the MicroPython firmware. This gives users the benefit of being able to access, read and write files within the main Python program for flexible storage operations.

There are two files that are treated specially by the ESP8266 when it starts up: *boot.py* and *main.py*. The *boot.py* script is executed first (if it exists) and once it completes, the *main.py* script is executed. You can create these files yourself and populate them with the code that you want to run when the device starts up. *boot.py* generally contains the boot parameters, like whether to have serial debugging on or off, which Wi-Fi AP to connect, the main code to run, etc. Looping stays in *main.py*, e.g., the logic for blinking LEDs.

It is also worth noting that microcontrollers are generally programmed in C, wherein direct registers are accessed and manipulated to use the peripherals of the microcontrollers. The firmware code is cross-compiled and built for the target microcontroller architecture, and is flashed using a suitable programmer. MicroPython abstracts all these steps, but the constraint here is the real-time feature of microcontrollers, in general. Hence, MicroPython is not suitable for strict real-time applications.

Boards supported by MicroPython

There are numerous boards/chipsets supported by MicroPython, and more are expected to do so in the near future as the project